Large-Scale and Multi-Structured Databases

PlayerHive

Sebastiano Pala

Syed Rafay Zia

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

UNIVERSITÀ DI PISA

CROSSLAB
Innovation for industry 4.0

Application Highlights

PlayerHive is a platform that helps gamers to explore and organize a catalog of their favorite titles while participating in a vast social network.

Main Features:

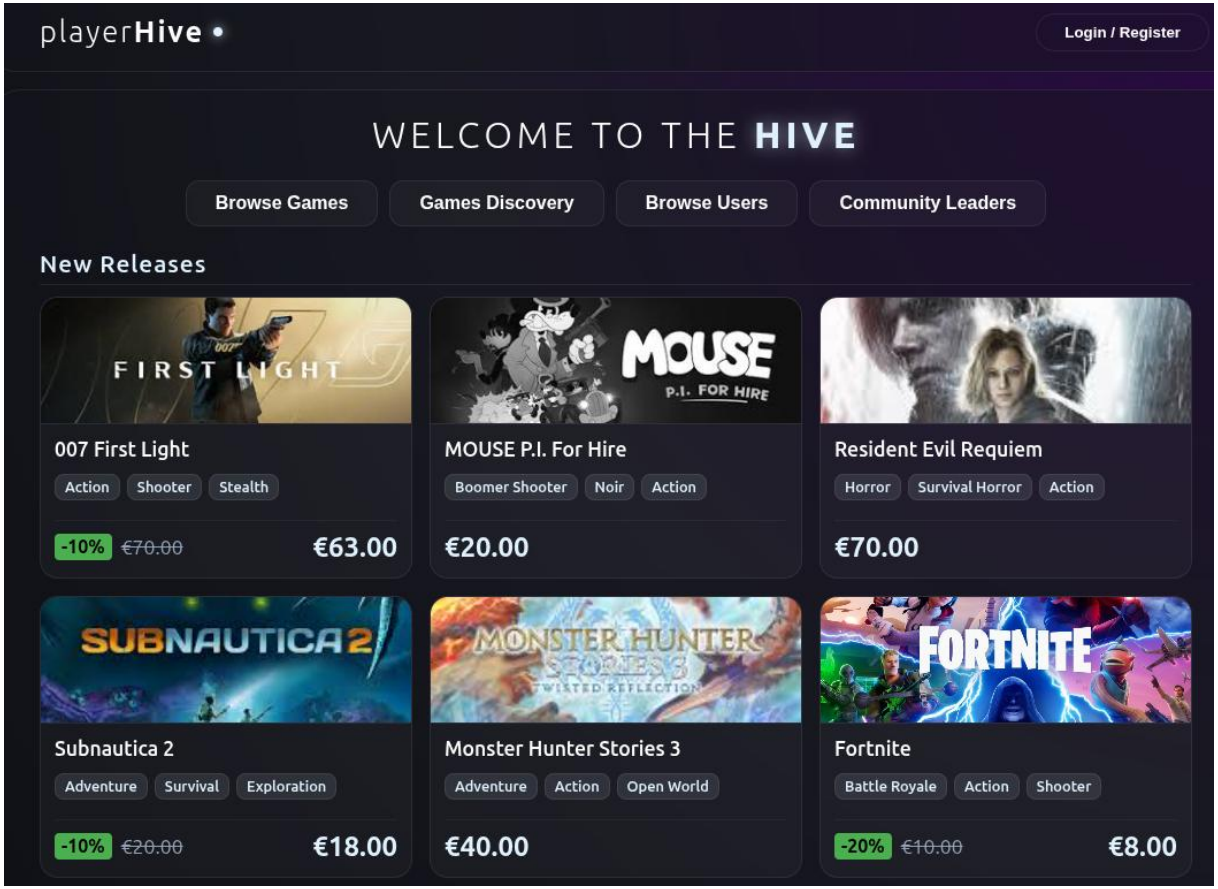
- Search within a complete and updated list of games and view their detailed info and community ratings
- Keep track of your progress by updating your personal library with playtime and unlocked achievements
- Rate and review your favorite titles to share your feedback with the community
- Make friends with other users to see what they are playing and discover their game libraries
- Receive Personal Games and Users Recommendations based on your own activity

Admin Features:

- Add or edit entries in the app's global game catalog
- Moderate the community by managing user profiles
- Gain insights such as genre performance and gaming trends

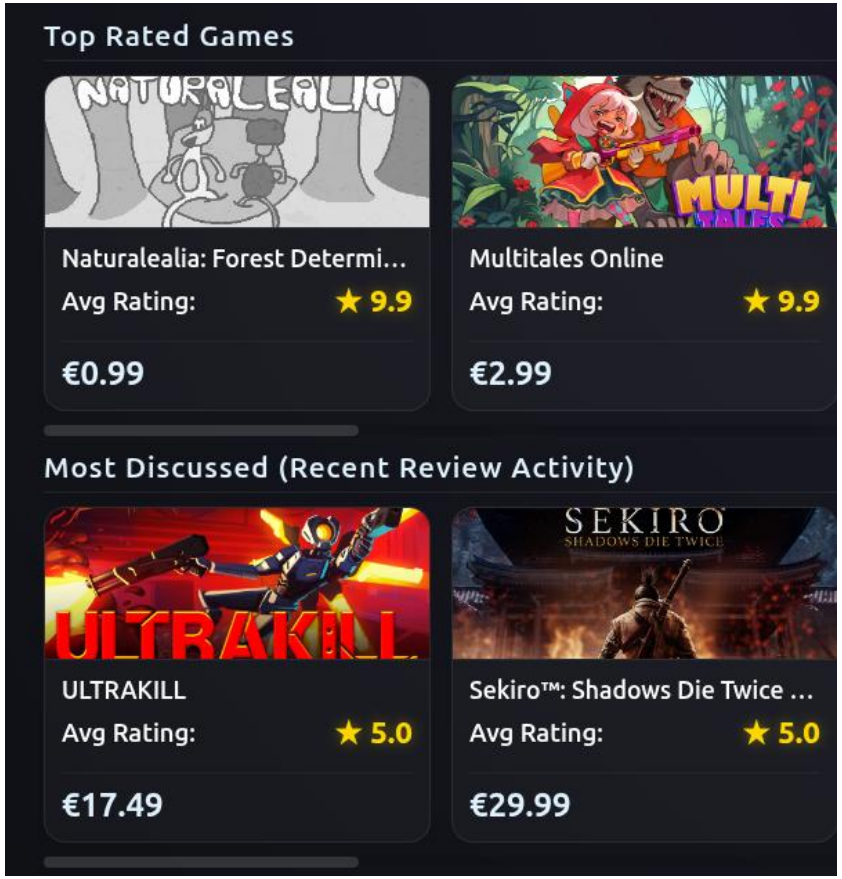
Actors and Mockups

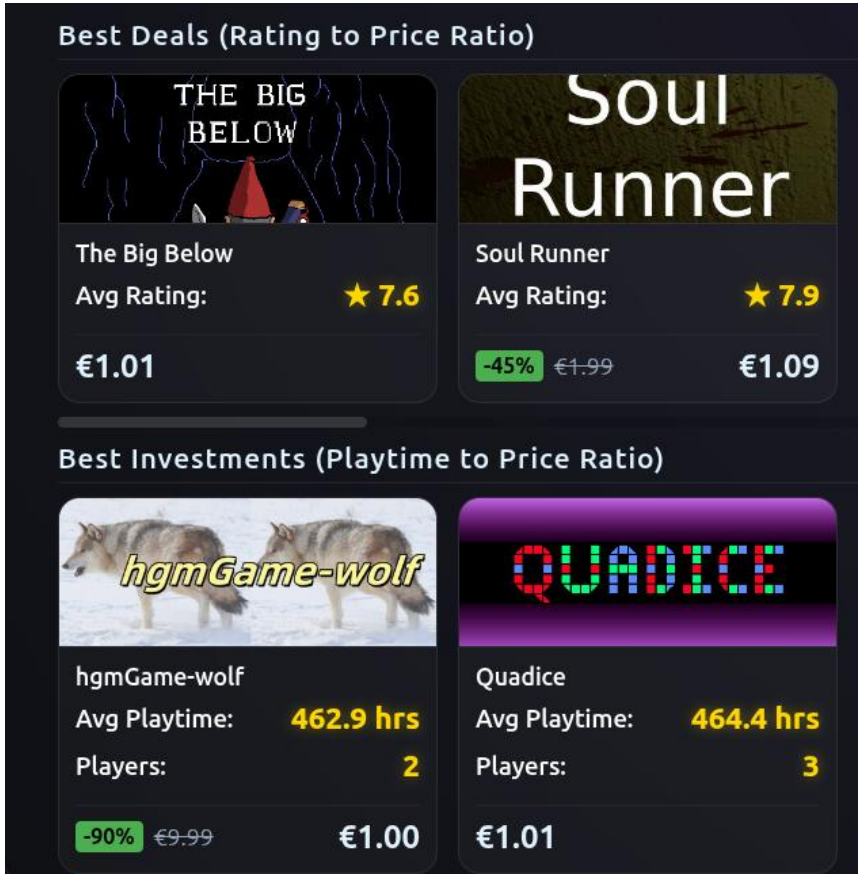
Unregistered User: Front Page



Actors and Mockups

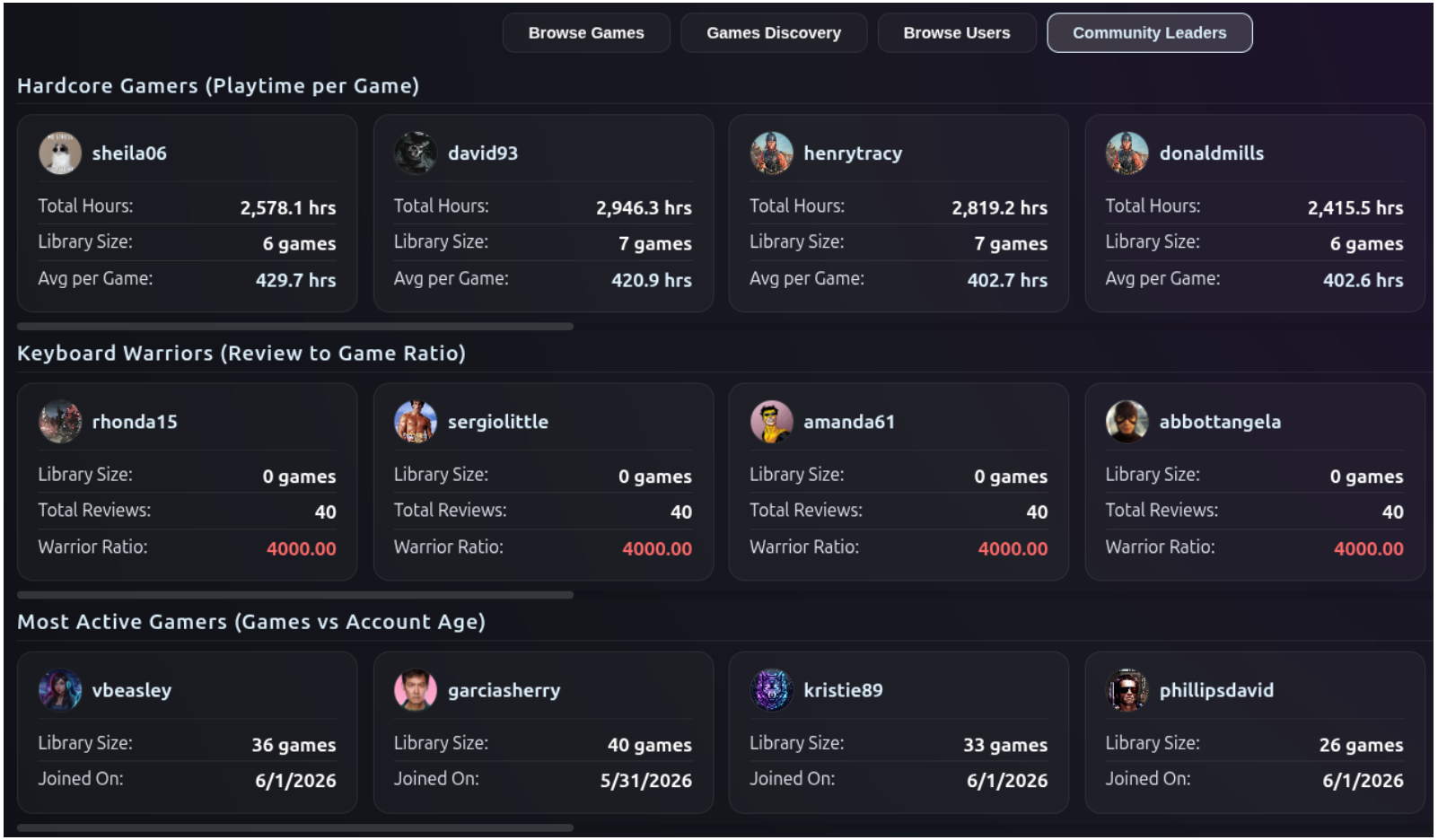
Unregistered User: Games Discovery





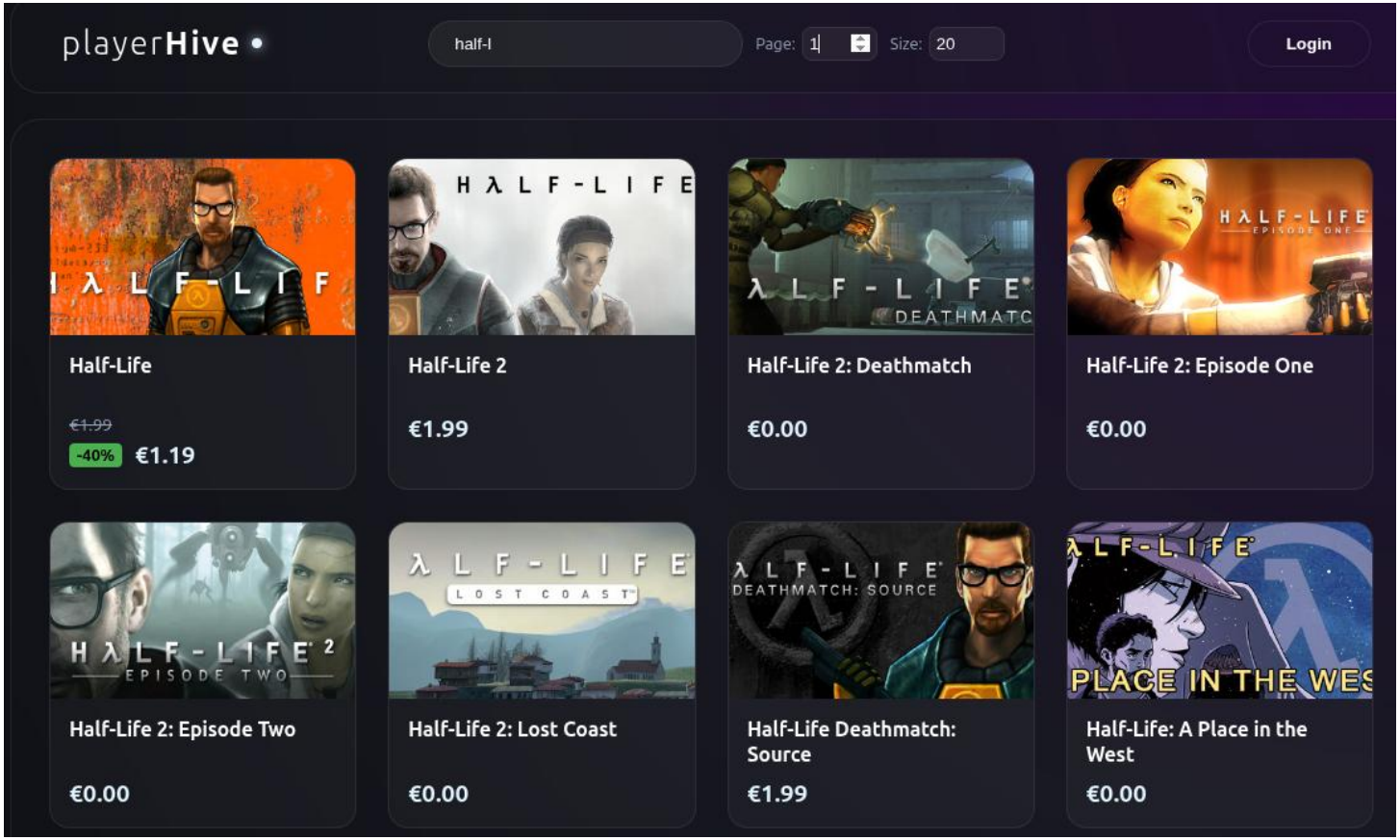
Actors and Mockups

Unregistered User: Community Leaders



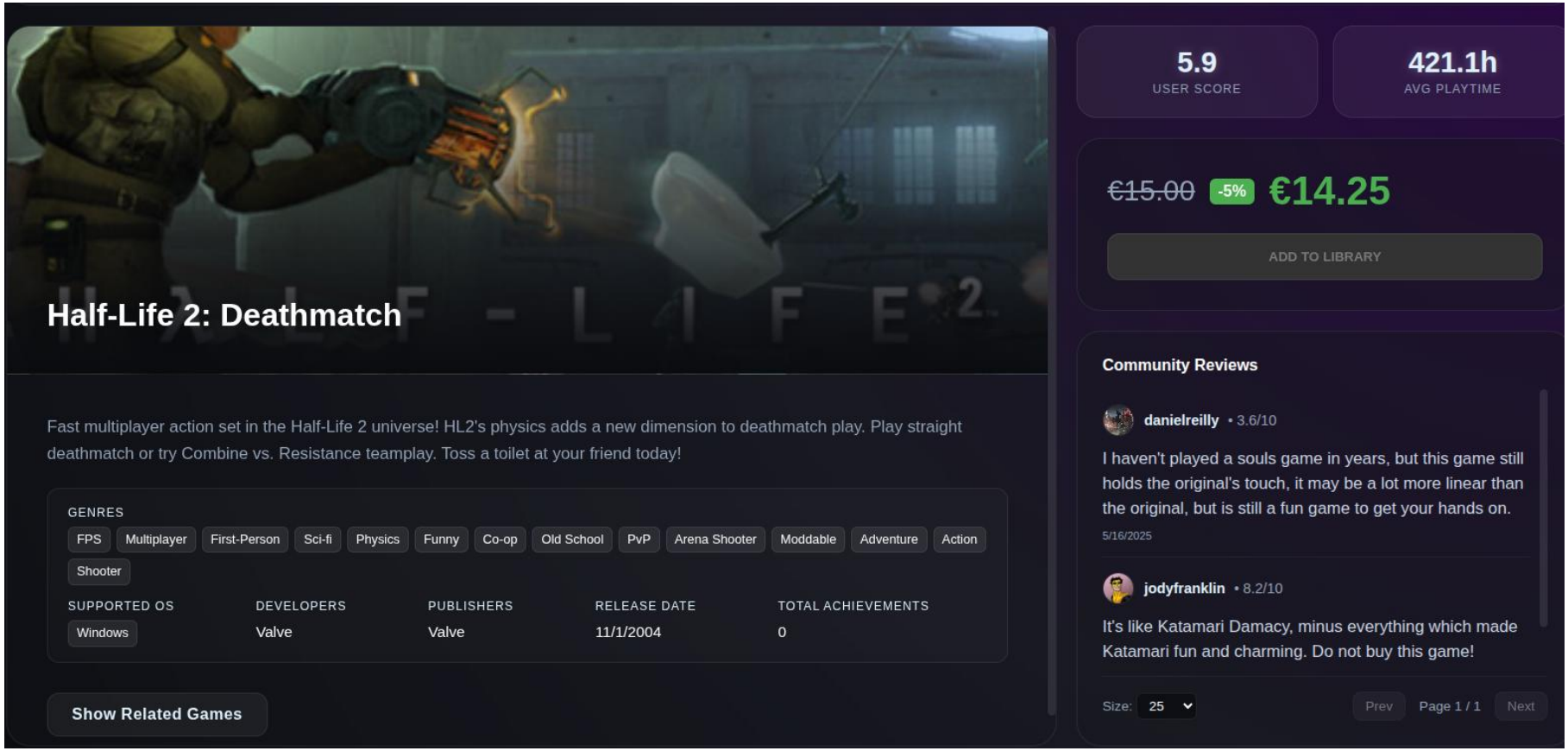
Actors and Mockups

Unregistered User: Game Search



Actors and Mockups

Unregistered User: Game Profile



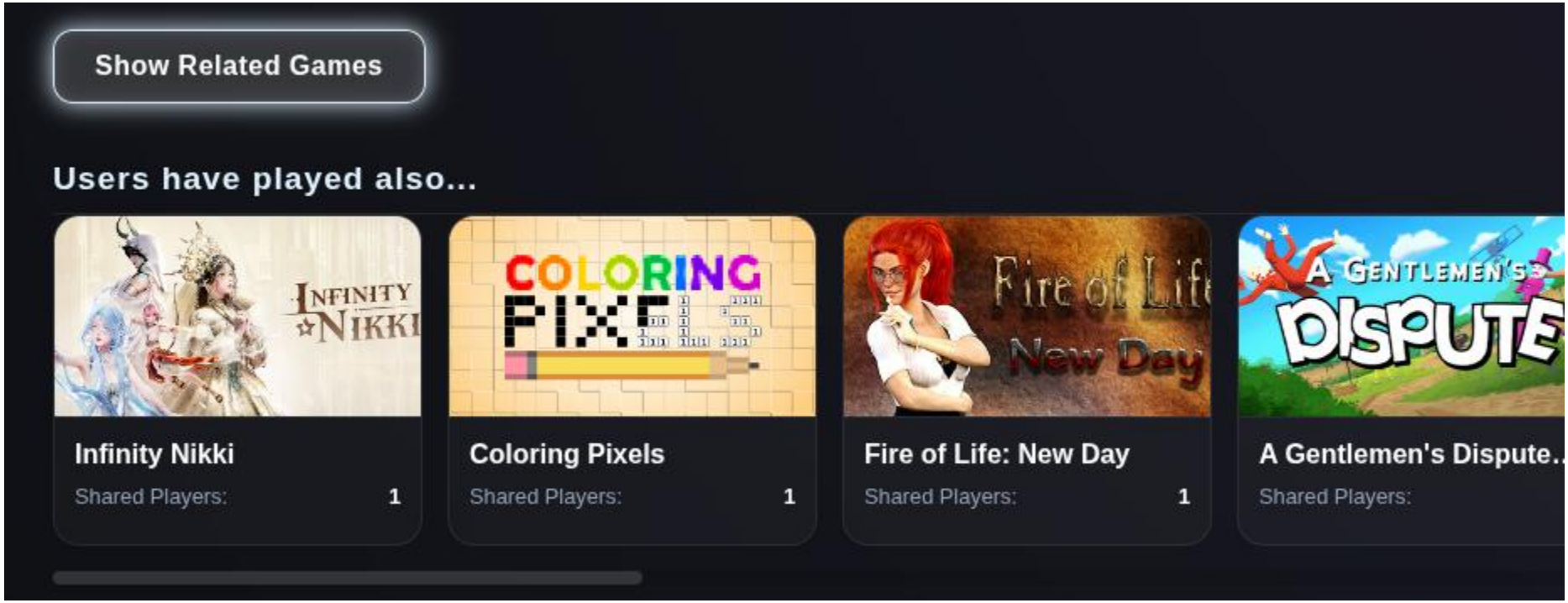
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

UNIVERSITÀ DI PISA

CROSSLAB Innovation for industry 4.0

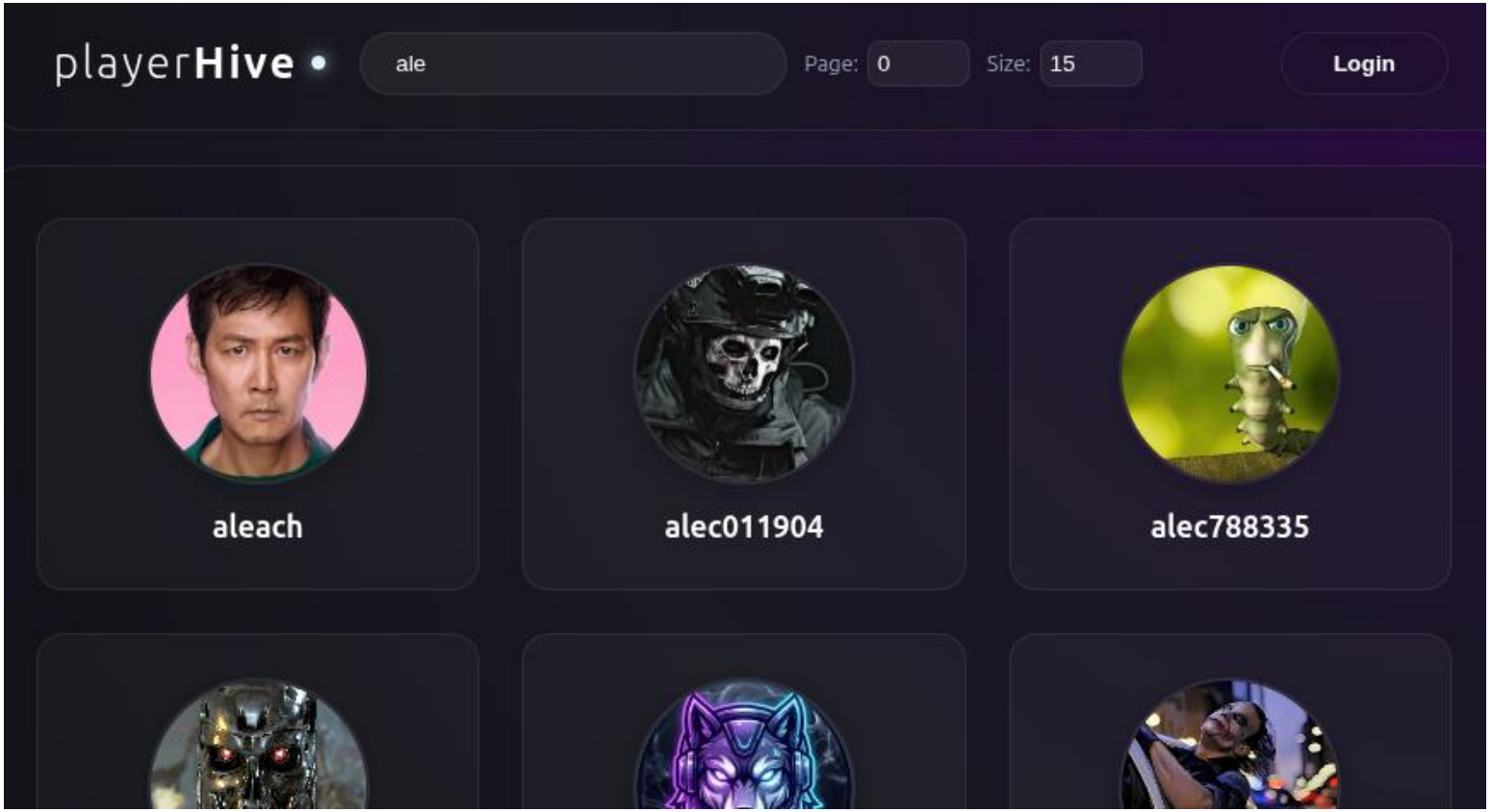
Actors and Mockups

Unregistered User: Related Games



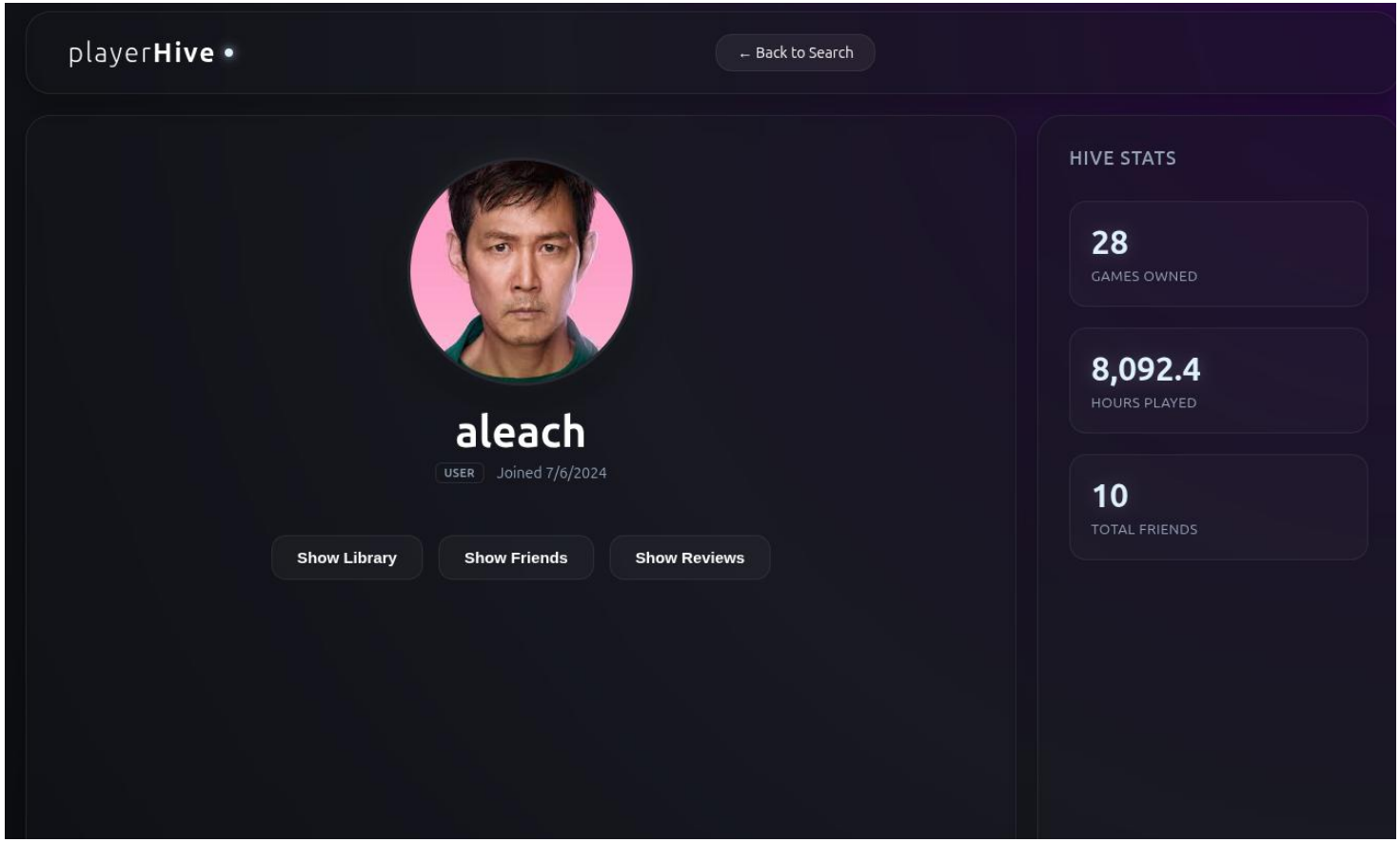
Actors and Mockups

Unregistered User: User Search



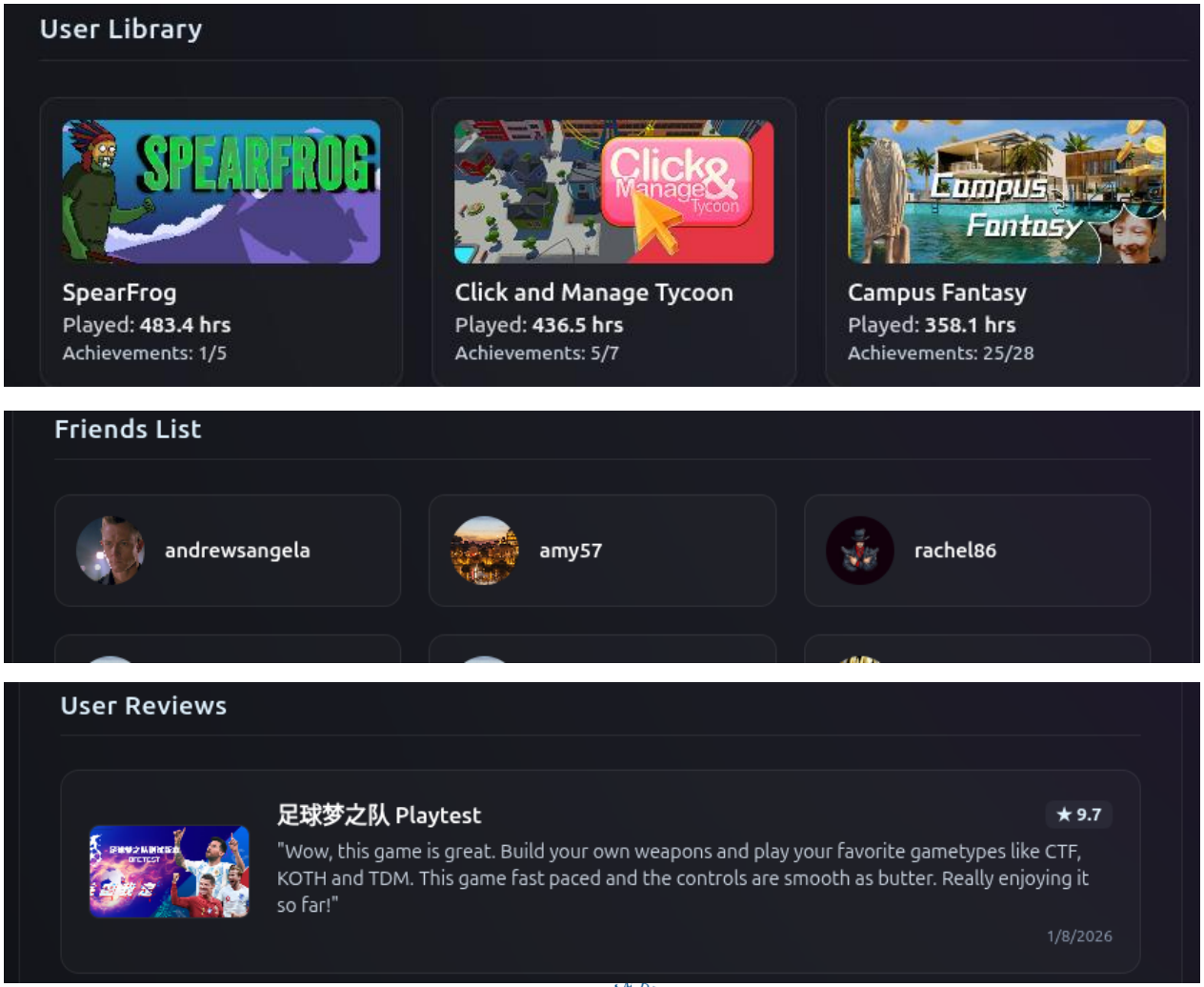
Actors and Mockups

Unregistered User: User Profile



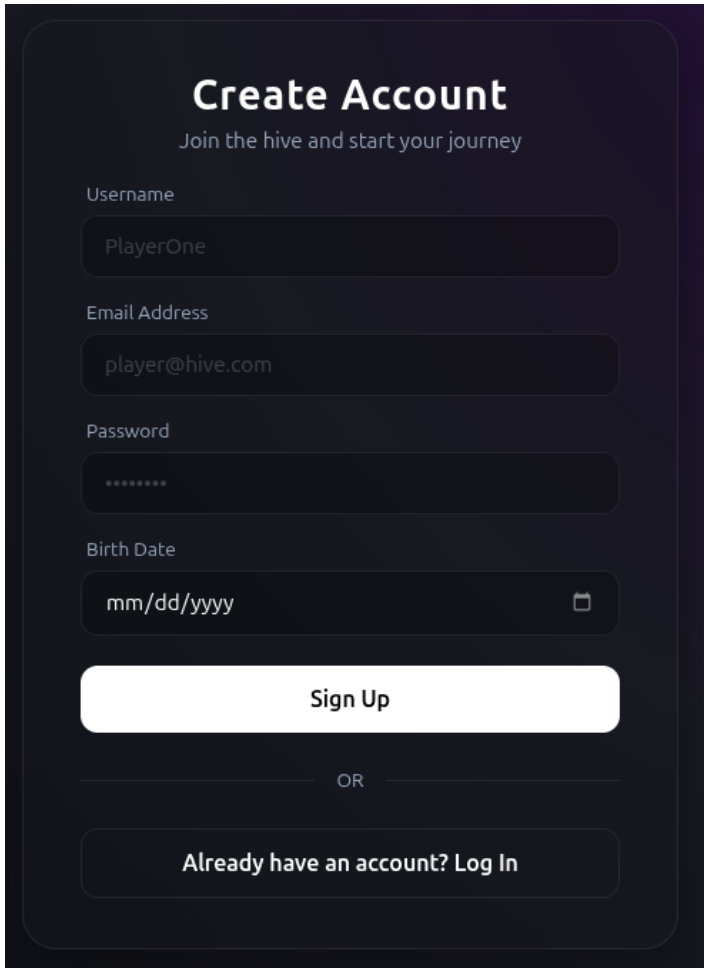
Actors and Mockups

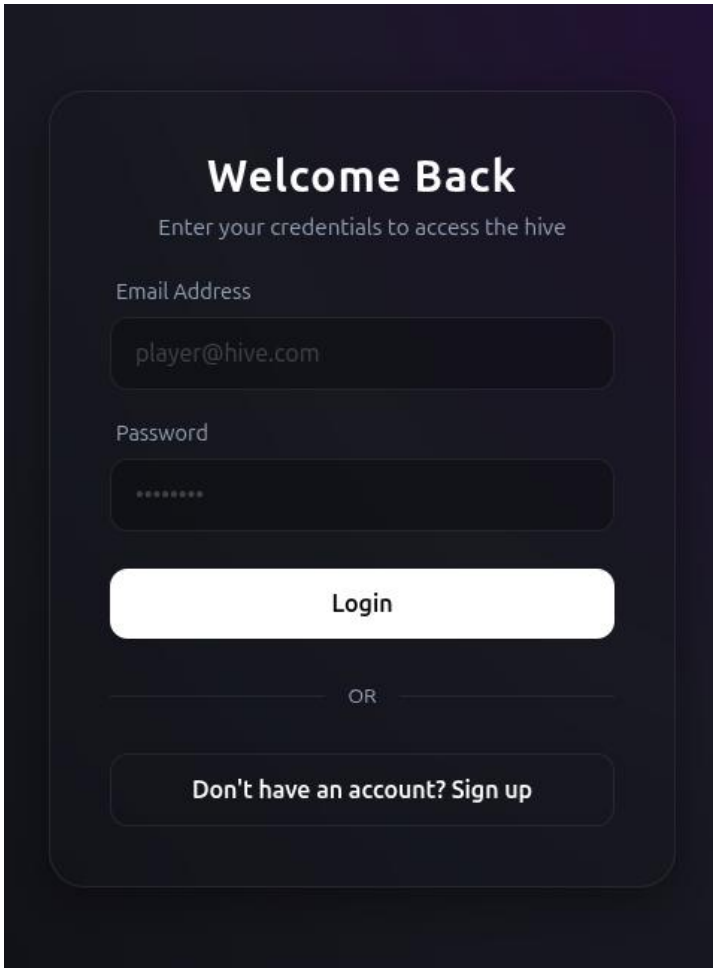
Unregistered User: User Profile in Detail



Actors and Mockups

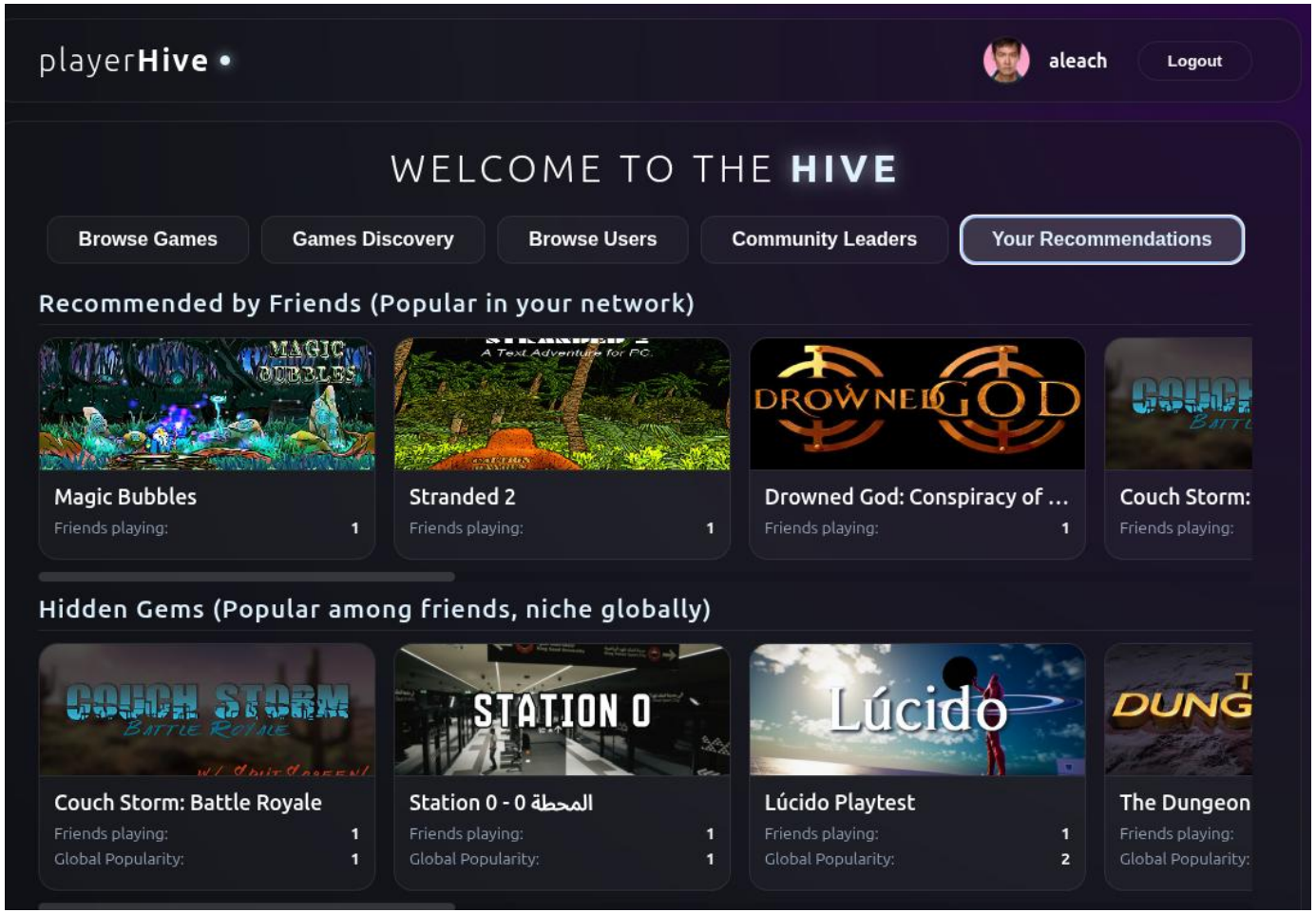
Unregistered User: Authentication





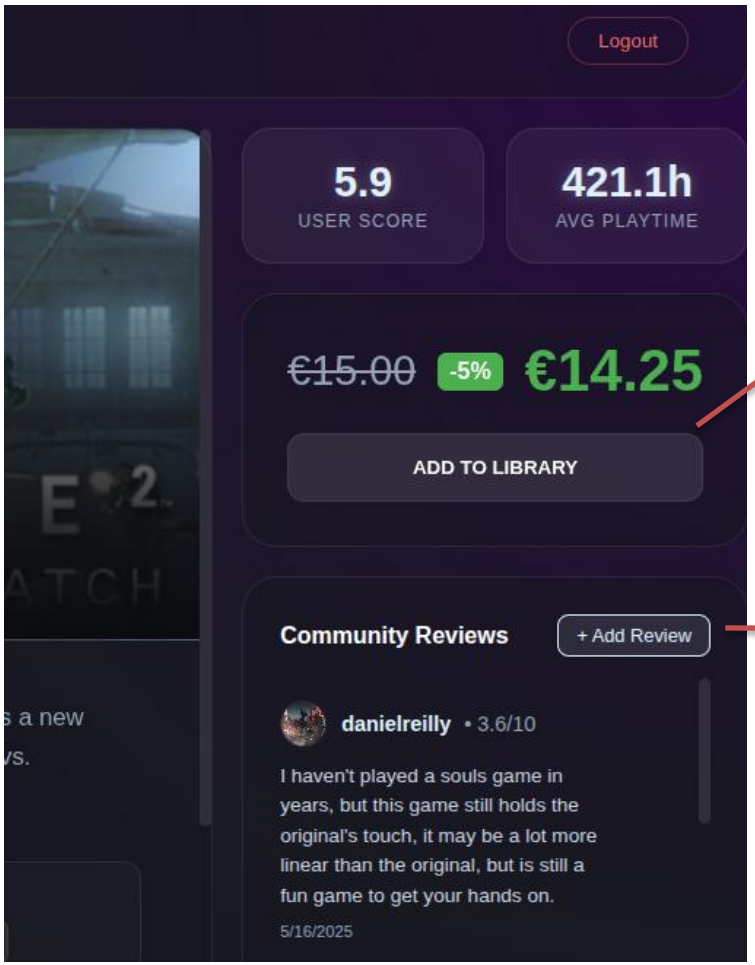
Actors and Mockups

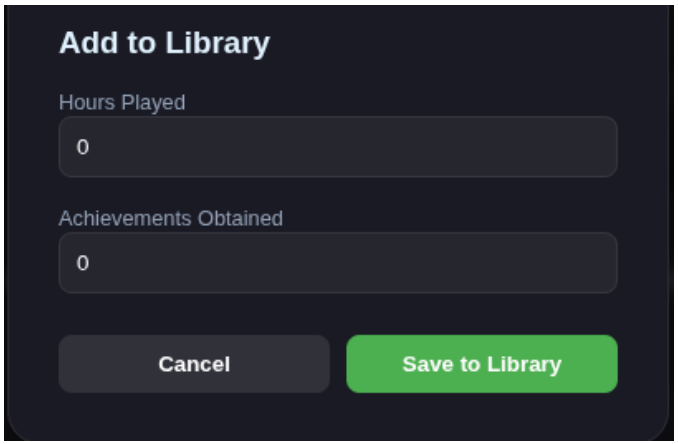
Registered User: Personal Recommendations

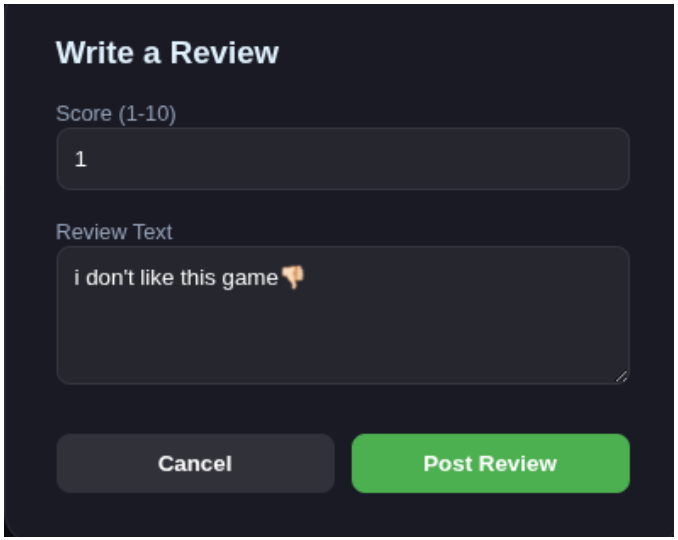


Actors and Mockups

Registered User: Game Profile

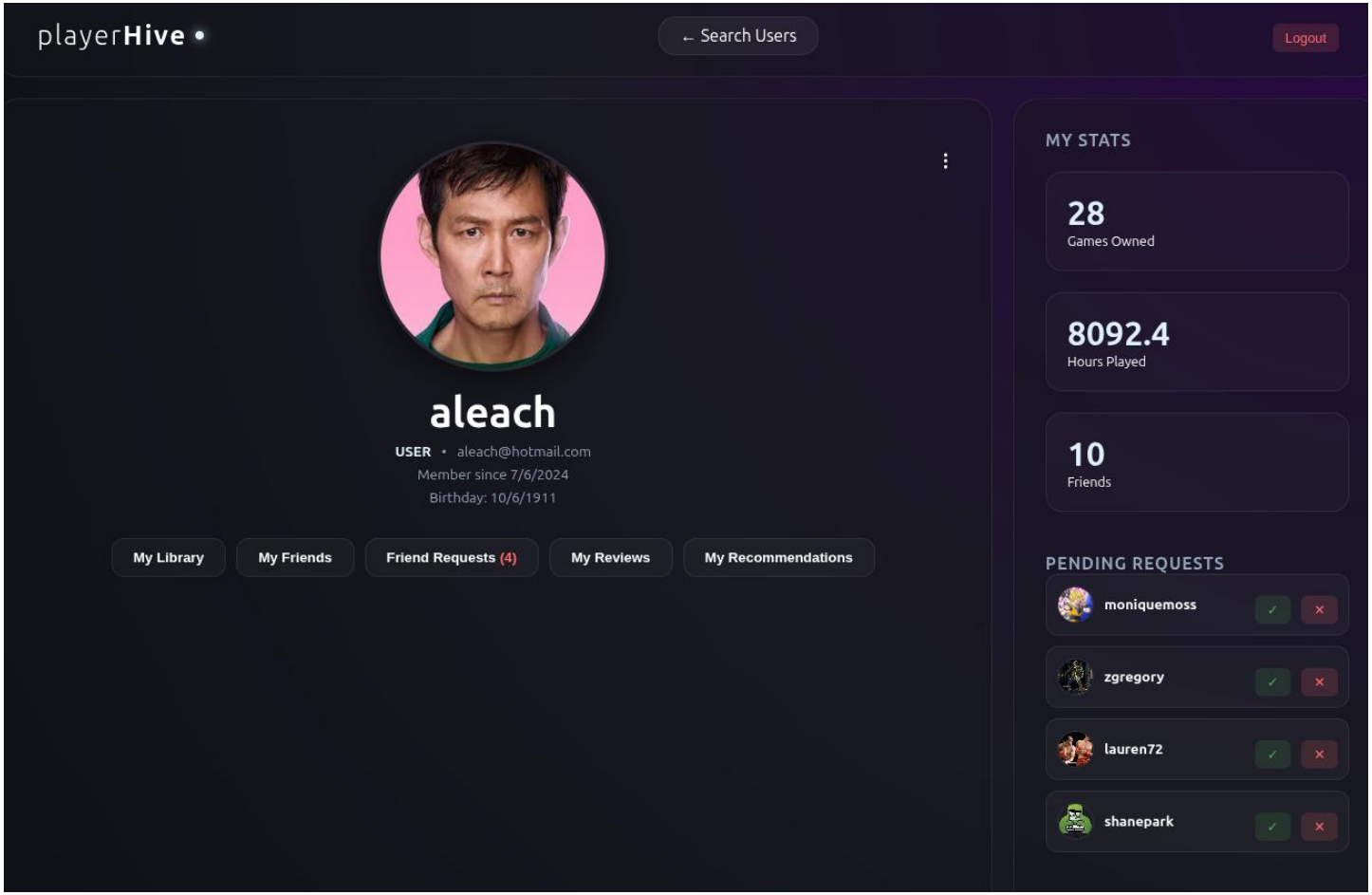






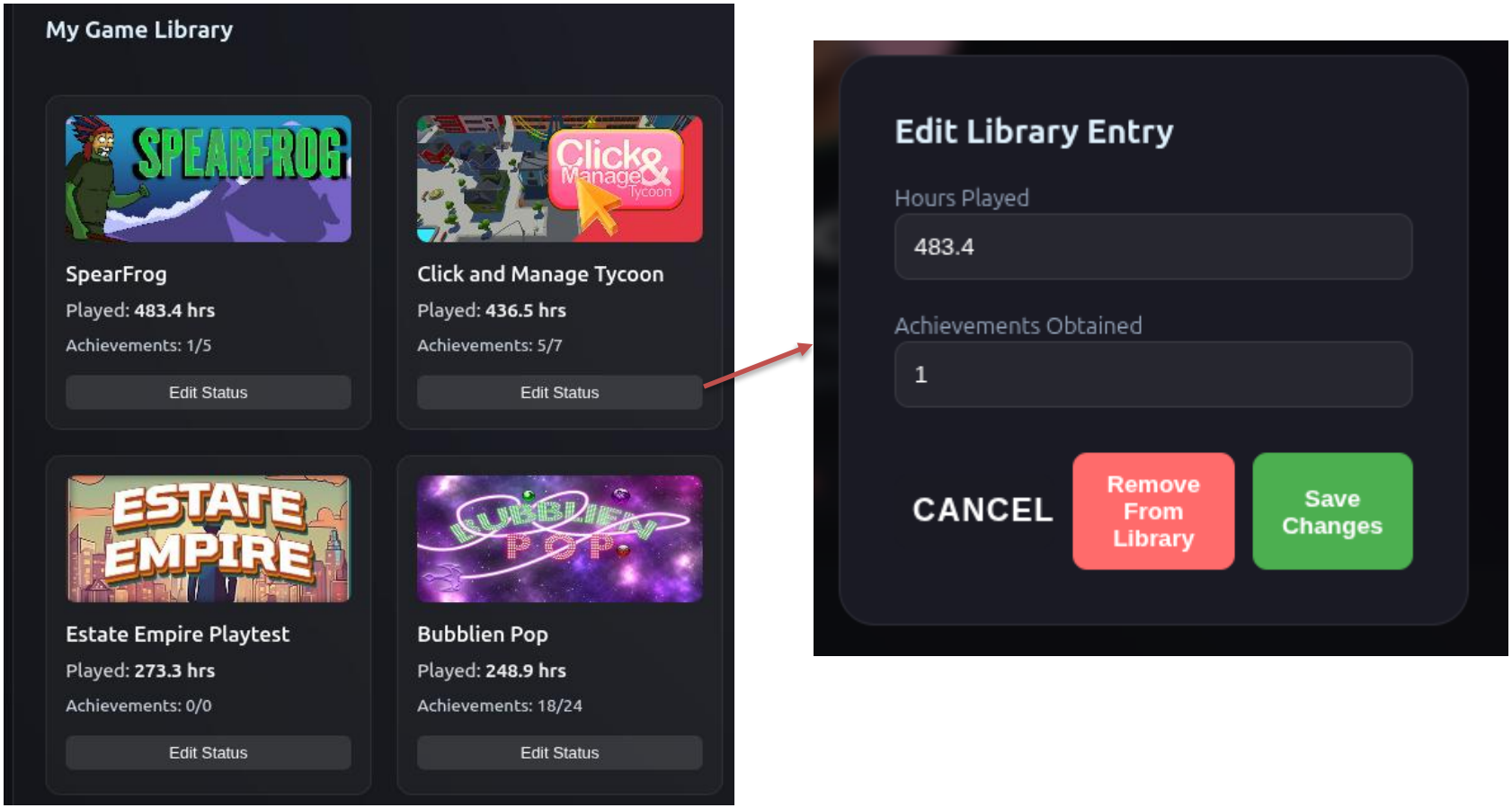
Actors and Mockups

Registered User: Own Profile



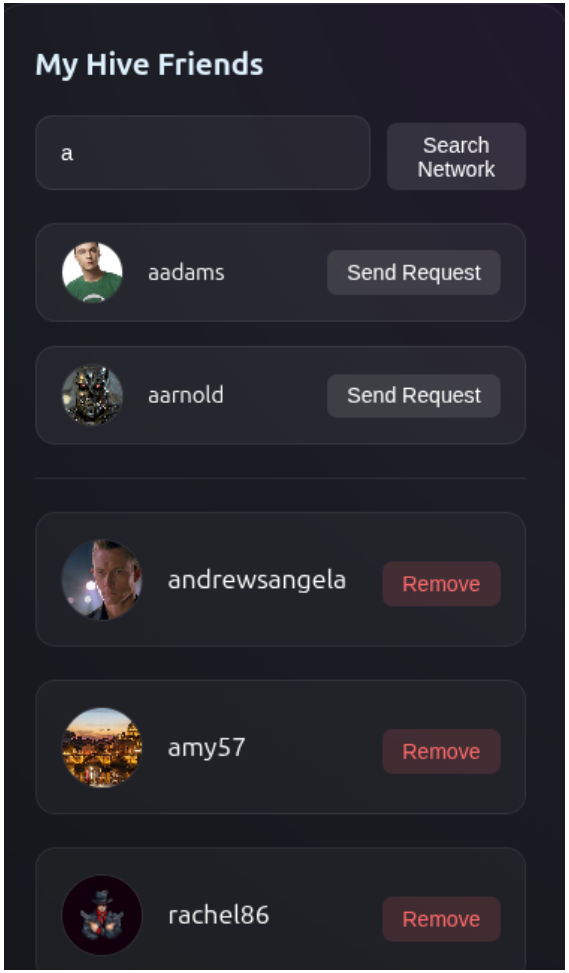
Actors and Mockups

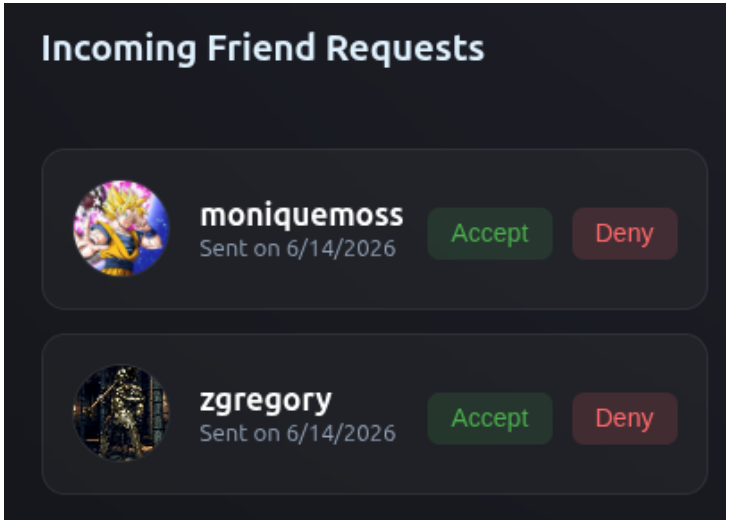
Registered User: Library Management

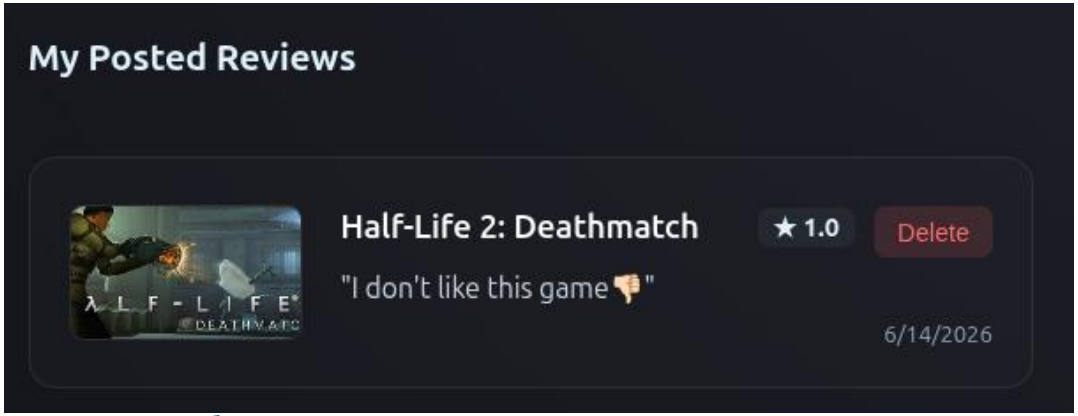


Actors and Mockups

Registered User: Friends and Reviews Management

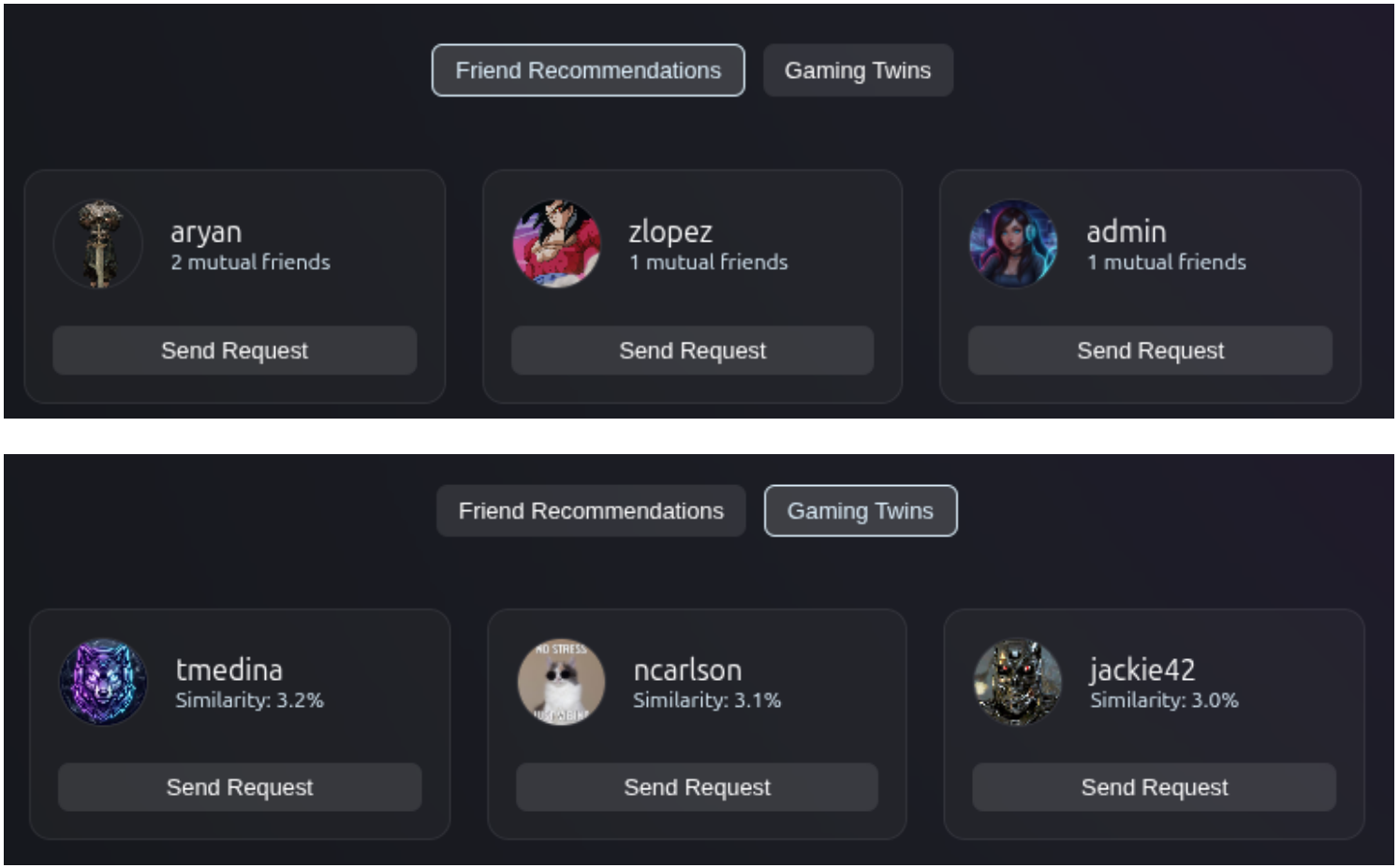






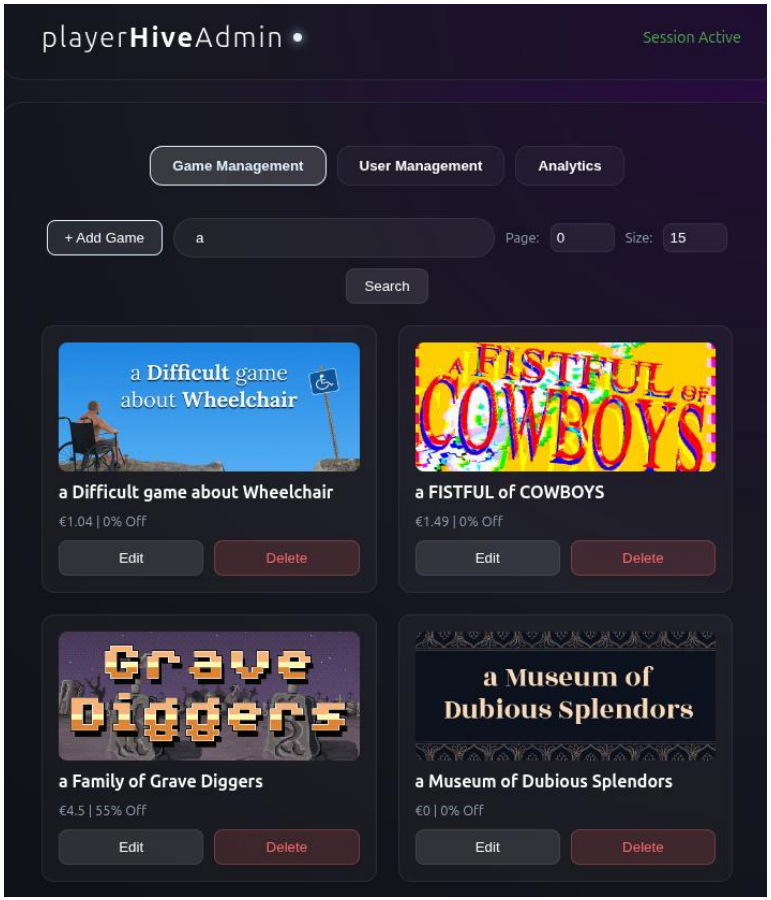
Actors and Mockups

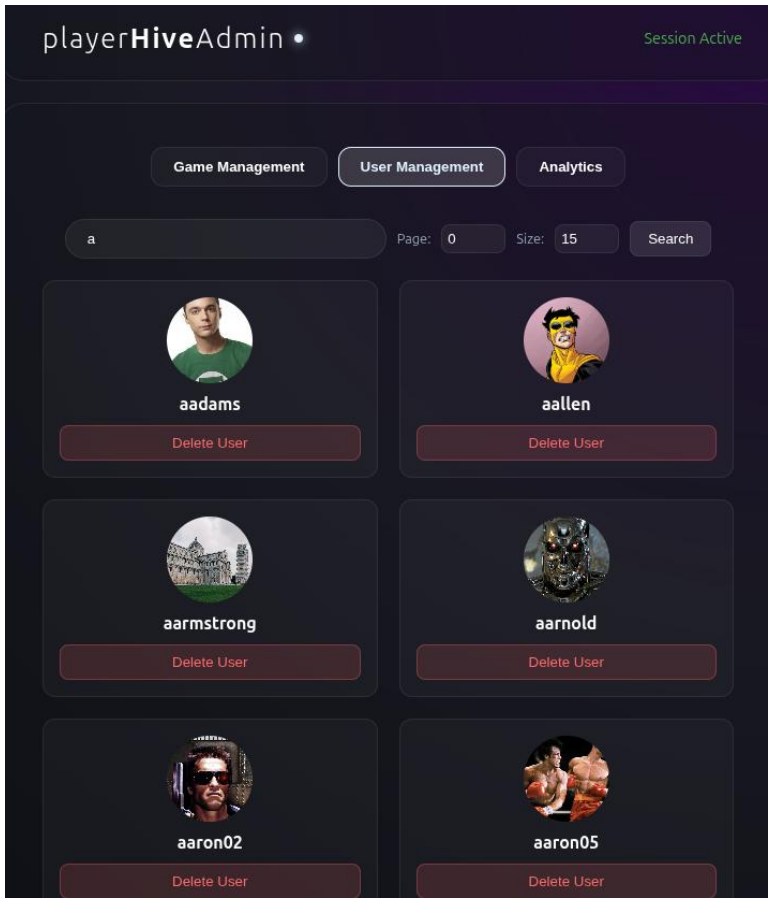
Registered User: Other User Recommendations



Actors and Mockups

Administrator: Game and User Management





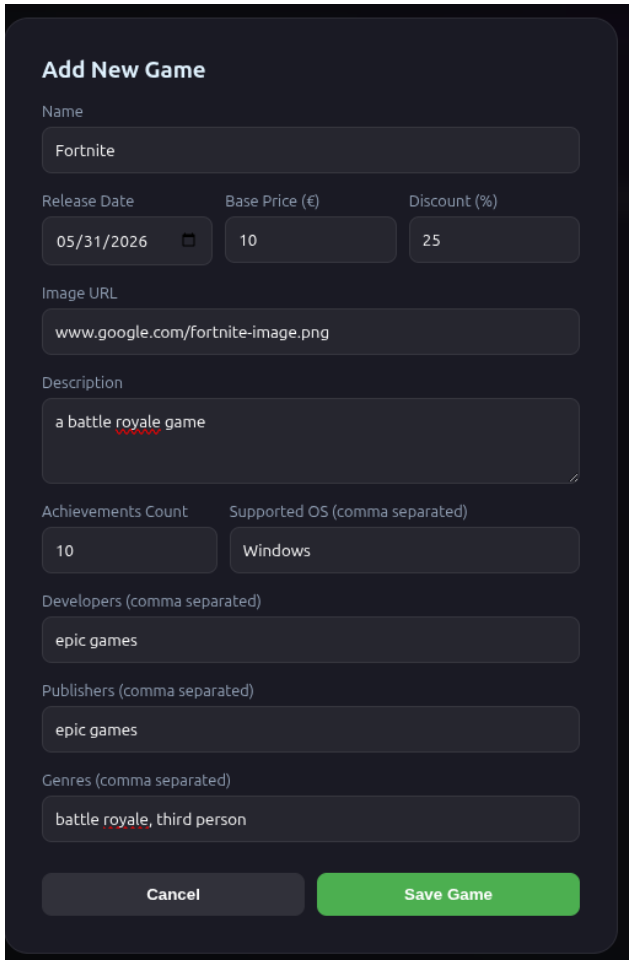
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

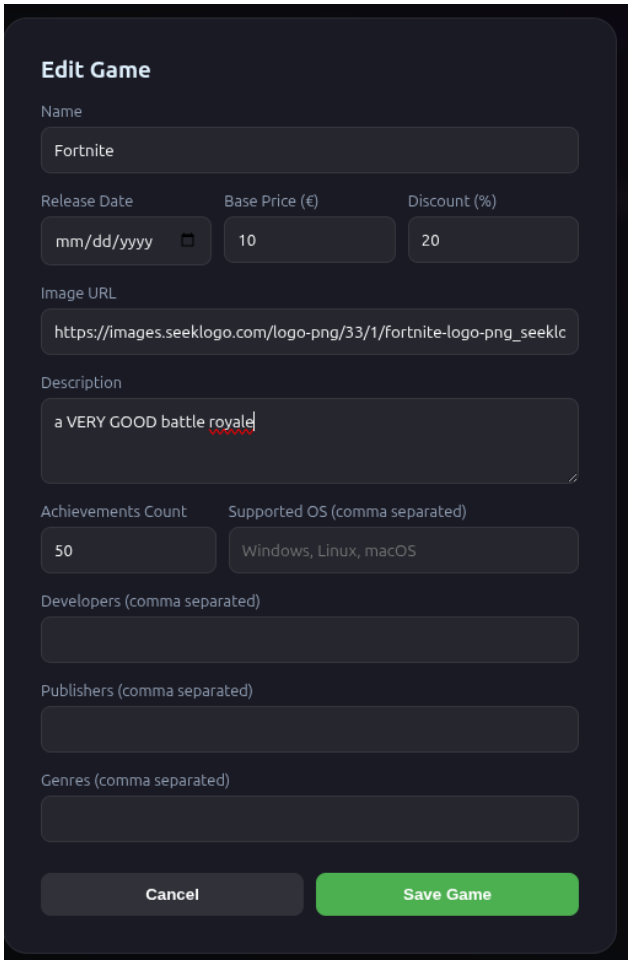
IN SUPREMÆ DIGNITATIS 1343
UNIVERSITÀ DI PISA

CROSSLAB Innovation for industry 4.0

Actors and Mockups

Administrator: Adding / Editing a Game





Actors and Mockups

Administrator: Analytics

Dataset Description

Sources: *Kaggle, Steam*

Description: *the data was scraped using* Steam store page scraping and **Steam Games Scraper** repository, which gathers data using the Steam API and Steam Spy.

Volume: 800 MB (Games), 2.2 GB (Reviews)

Variety: Two different datasets were used to increase variety. The data itself was scraped using different APIs.

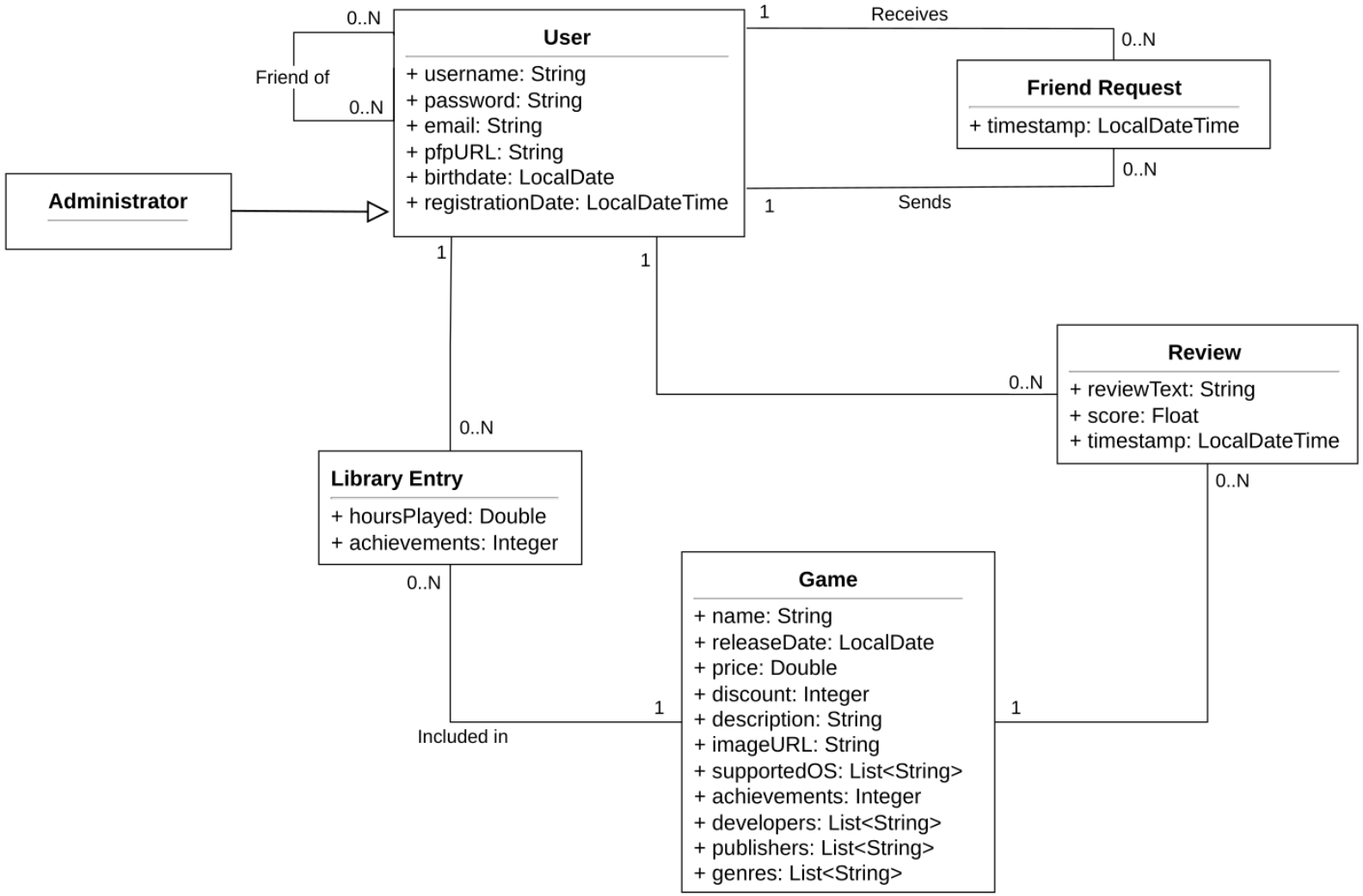
Velocity/Variability: The data changes very fast. The users' game libraries are in continuous evolution, and game scores can change based on the amount of attention a game receives.

Dataset Description

Pre-processing: A python script was used to process the data. The script manages:

- Cleaning of the data, both for games and reviews
- Generation of 15.000 synthetic users, an admin profile, and a static user
- Generation of each user's friendships and libraries
- Association of a random number of reviews to each game
- Creation of the indexes on MongoDB and Neo4j
- Uploading the results to the databases

UML Design Class Diagram



Application non-functional requirements

Performance

- The system must be read-focused, prioritizing the performance of the most frequent read operations.
- The system must paginate all list-based endpoints' responses to prevent over-fetching.

Availability and Scalability

- The system must prioritize Availability over data Consistency, accepting eventual consistency as a tradeoff.
- The system must be capable of accommodating growth in user traffic without degrading response time.

Implementation

- The system must implement a polyglot persistence architecture utilizing both a DocumentDB and a GraphDB.
- The system must be developed in Java using the Spring Boot framework.
- The system must expose all functionality through RESTful API endpoints.

Security

- The system must have a stateless authentication.
- The system must store user passwords as BCrypt hashes.

Document DB Design

User collection


```
{
  "_id": {
    "$oid": "663a1f2b4e3d7a0012c4b891"
  },
  "username": "ProGamer99",
  "email": "progamer99@hotmail.com",
  "password": "$2a$04$...",
  "role": "USER",
  "pfpURL": "/Playerhive/pfp/a1b2c3d4",
  "birthdate": {
    "$date": "1998-06-14T00:00:00.000Z"
  },
  "registrationDate": {
    "$date": "2023-09-01T10:15:00.000Z"
  },
  "numGames": 5,
  "hoursPlayed": 210.3,
  "friends": 3,
  "friendRequests": [
    {
      "user_id": {
        "$oid": "663a2c1d4e3d7a0012c4b900"
      },
      "username": "SpeedRunner42",
      "pfpURL": "/Playerhive/pfp/e5f6g7h8",
      "timestamp": {
        "$date": "2025-04-20T14:32:00.000Z"
      }
    }
  ],
  "requestsNum": 1,
  "reviewIds": [
    {
      "review_id": {
        "$oid": "663b0a0f4e3d7a0012c4c100"
      },
      "game_id": {
        "$oid": "663b0f014e3d7a0012c4c200"
      }
    }
  ]
}
```


Games collection


```
{
  "_id": {
    "$oid": "663b0f014e3d7a0012c4c200"
  },
  "name": "Elden Ring",
  "release_date": {
    "$date": "2022-02-25T00:00:00.000Z"
  },
  "price": 59.99,
  "discount": 10,
  "finalPrice": 53.99,
  "description": "A fantasy action-RPG adventure...",
  "image": "https://cdn.example.com/eldenring.jpg",
  "supportedOS": ["windows"],
  "achievements": 42,
  "developers": ["FromSoftware"],
  "publishers": ["Bandai Namco"],
  "genres": ["Action", "RPG", "Open World"],
  "sumScore": 463.5,
  "countScore": 52,
  "totalHoursPlayed": 12500.0,
  "numPlayers": 47,
  "recentReviews": [
    {
      "_id": {
        "$oid": "541a3d7d0e3f4fb786a8f41c"
      },
      "user_id": {
        "$oid": "6f2c9a746e444ef79b311db7"
      },
      "username": "icollier",
      "pfpURL": "/Playerhive/pfp/1acf31...",
      "review_text": "Do not buy this game.",
      "score": 2,
      "timestamp": {
        "$date": "2025-11-08T00:21:29.000Z"
      }
    }
  ],
  "allReviews": [
    {
      "$oid": "663b0a0f4e3d7a0012c4c100"
    },
    {
      "$oid": "663b0a0f4e3d7a0012c4c101"
    }
  ]
}
```


Reviews Collection


```
{
  "_id": {
    "$oid": "541a3d7d0e3f4fb786a8f41c"
  },
  "game_id": {
    "$oid": "18f62c43c8304d9187677504"
  },
  "user_id": {
    "$oid": "6f2c9a746e444ef79b311db7"
  },
  "username": "icollier",
  "pfpURL": "/Playerhive/pfp/1acf31ec3a2...",
  "game_name": "Red Barrels Go Boom",
  "game_image": "https://shared.akamai.st...",
  "review_text": "Do not buy this game.",
  "score": 2,
  "timestamp": {
    "$date": "2025-11-08T00:21:29.000Z"
  }
}
```

Document DB Design

Get Hardcore Gamers

Objective: Identify the most dedicated players on the platform based on their library size and overall engagement.

Implementation: The aggregation pipeline isolates highly engaged users and computes their average hours per game to rank their depth of engagement.

```
db.user.aggregate([

  { $match: { numGames: { $gt: ?minGames },
    hoursPlayed: { $gt: ?minHours } } },

  { $project: {
      _id: 1,
      username: 1,
      pfpURL: 1,
      totalHours: '$hoursPlayed',
      numGames: 1,
      avgHoursPerGame: {
        $round: [
            {
                $divide: [
                    '$hoursPlayed', '$numGames'
                        ]
            }, 1
        ]
      }
  }},

  { $sort: { avgHoursPerGame: -1 } },

  { $limit: 15 }
])
```

Document DB Design

Genres Stats (Analytic)

Objective: Reveal whether critically acclaimed genres also retain the most player engagement (hours played).

Implementation: Utilizes an aggregation pipeline to deconstruct the genres array, grouping titles to compute the average review score, mean playtime per player, and total game count per category.

```
db.game.aggregate([

  { $match: { countScore: { $gt: 0 },
              numPlayers: { $gt: 0 } }
              },

  { $project: {
      genres: 1,
      avgScore: { $divide: ['$sumScore',
                            '$countScore'] },
      avgHoursPerPlayer: { $divide: ['$totalHoursPlayed',
                                     '$numPlayers'] }
  }},

  { $unwind: '$genres' },

  { $group: {
      _id: '$genres',
      avgScore: { $avg: '$avgScore' },
      avgHoursPerPlayer: { $avg: '$avgHoursPerPlayer' },
      totalGames: { $sum: 1 }
  }},

  { $sort: { avgScore: -1 } }
])
```

Document DB Design

Release Year Stats (Analytic)

Objective: Track the historical evolution of game quality

Implementation: Utilizes an aggregation pipeline to extract the release year from date objects, to compute the average review score and total game volume per year

```
db.game.aggregate([

  { $match: { countScore: { $gt: 0 },
              release_date: { $ne: null } }
              },

  { $project: {
      releaseYear: { $year: '$release_date' },
      avgScore: { $divide: ['$sumScore',
                            '$countScore'] }
  }},

  { $group: {
      _id: '$releaseYear',
      avgScore: { $avg: '$avgScore' },
      totalGames: { $sum: 1 }
  }},

  { $sort: { _id: 1 } }
])
```

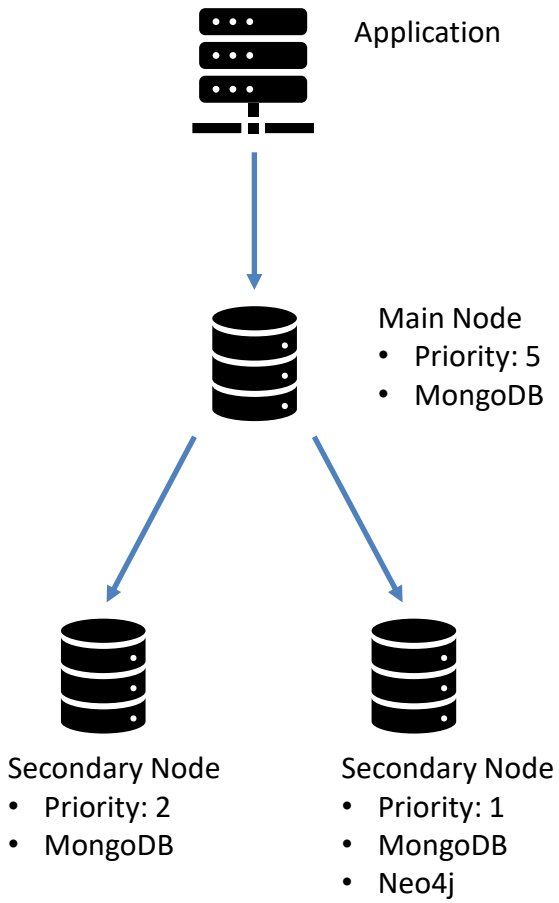
MongoDB Replica Set Configuration

Three-Node Configuration:

- The deployment utilizes a three-node replica set with asymmetric election priorities to ensure predictable behavior and resource optimization.
- Each node has a **different priority** (5,2,1), with the main node having priority 5.
- When (and if) the main node fails, the node with priority 2 will become the main node.
- The node with the lowest priority (1) also co-hosts the neo4j instance, and its low priority prevents this node from becoming Primary, which avoids hardware resource contention between MongoDB write operations and Neo4j graph traversals.

Availability Strategy:

This hierarchical voting mechanism ensures that even during a partition, the system can elect a leader quickly, adhering to the **AP (Availability and Partition Tolerance)** model.



MongoDB Replica Set Configuration

Write Concern (w: 1)
Write operations are acknowledged as soon as they are committed to the Primary node. This configuration minimizes latency by bypassing the wait for secondary replication, thereby supporting the high throughput required for intensive social interactions. By prioritizing system uptime and low-latency access over strict real-time consistency, this approach perfectly matches the platform's social and entertainment-centric nature while drastically reducing round-trip times for a smoother overall user experience.

Read Preference: nearest
Read queries are dynamically routed to the replica set member with the lowest network latency. This strategy effectively distributes the read load across the entire cluster and reduces pressure on the Primary node. Consequently, it ensures minimal response times for users and provides the scalability needed to seamlessly accommodate traffic growth without degrading system performance.

MongoDB Indexes

USED:

- **Unique B-tree Indexes:**
  - **Users collection:** "*email*" and "*username*"
  - **Games collection:** "*name*"
- **Descending Index:**
  - **Games collection:** "*release_date*"

REASON:

- Unique indexes enforce data consistency during registration and provide lookups for authentication.
- The name and username indexes support anchored regexp queries, enabling fast catalog and user browsing without full collection scans.
- The release_date index allows for a fast-loading front page, removing the need for a collection scan and an in-memory sort

NOT USED:

- We did not define any indexes for the reviews collection.
- Access is handled exclusively via *_id* arrays embedded in parent documents.

MongoDB Indexes

Index	State	Plan stage	Docs examined	Time (ms)
users.email	without	COLLSCAN	15,002	40
	with	EXPRESS_IXSCAN	1	1
users.username	without	COLLSCAN	15,002	36
	with	EXPRESS_IXSCAN	1	1
users.username (search)	without	COLLSCAN	15,002	39
	with	FETCH+IXSCAN	22	4
games.name	without	COLLSCAN	121,091	767
	with	EXPRESS_IXSCAN	1	2
games.name (search)	without	COLLSCAN	121,091	803
	with	FETCH+IXSCAN	10	3
games.release_date	without	SORT+COLLSCAN	121,091	727
	with	LIMIT+FETCH+IXSCAN	15	2

Discussion on MongoDB Data Sharding

Users and Games

- **Algorithm chosen:** Hashed Sharding by *_id*
- **Logic:** Data is distributed across shards using a hashed unique identifier.
- **Functional Benefit:** Optimizes the most frequent access pattern (fetching specific profiles or game details via ID).
- **Non-Functional Benefit:** Ensures uniform data distribution and prevents hardware hotspots, allowing the system to scale horizontally as user traffic grows.

Discussion on MongoDB Data Sharding

Reviews

Algorithm chosen: (Targeted Sharding by *game_id*)

Logic: All reviews for a specific title are physically co-located on the same shard.

Functional Benefit: Aligns with the core user journey; fetching a game's review history becomes a targeted single-shard operation rather than a broadcast query.

Non-Functional Benefit: Drastically reduces network overhead and query latency, prioritizing high availability and rapid response times for the platform's most common read path.

Architectural Trade-offs: While sharding reviews by *game_id* makes user-centric lookups more expensive (scatter-gather), it fulfills the non-functional requirement of prioritizing the most critical read operations at scale.

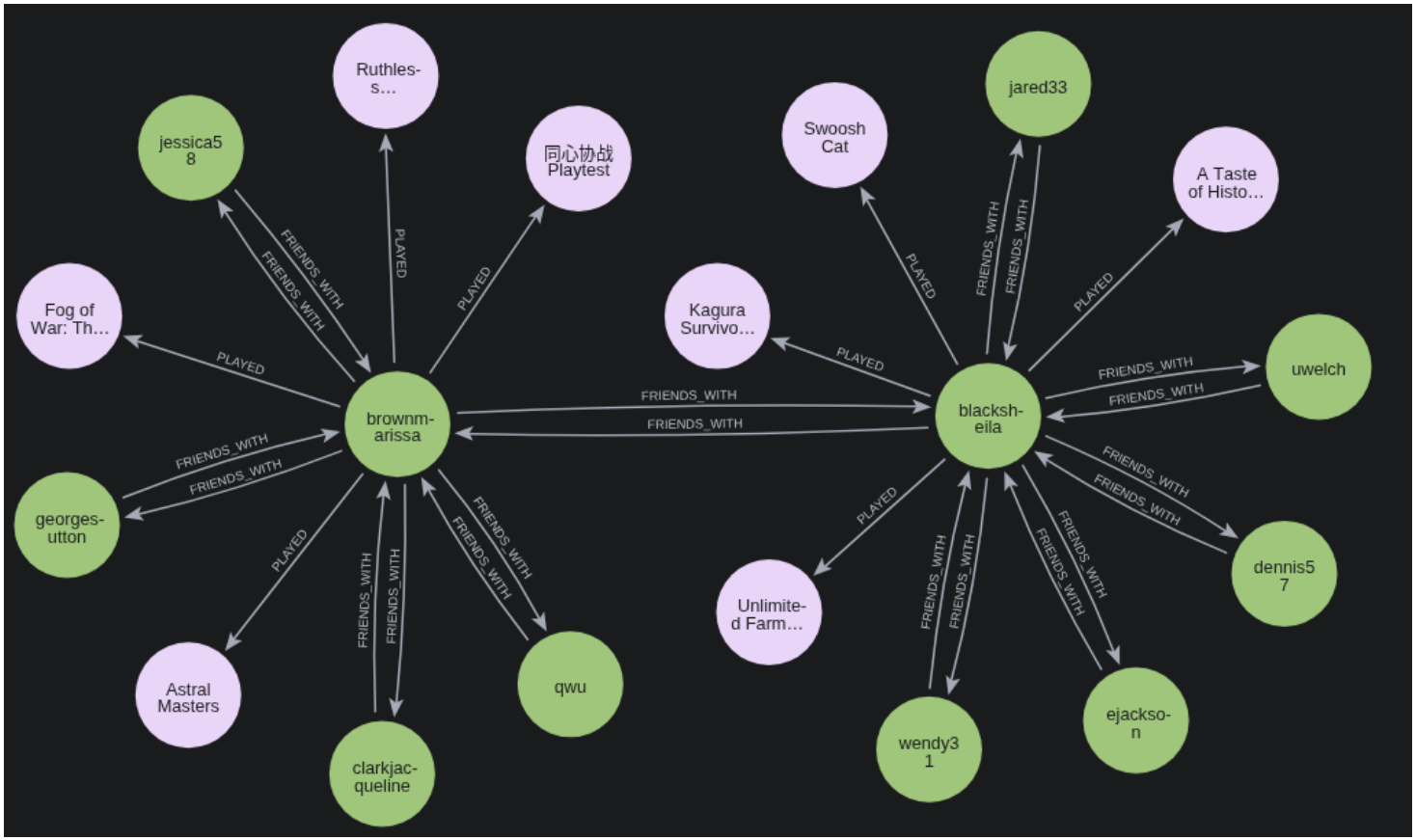
Graph DB Design

Nodes:

- :User{id,username,pfpURL}
- :Game{id, name, image, achievements}

Relationships:

- :User <– [:FRIENDS_WITH] -> :User
- :User – [:PLAYED{hoursPlayed,achievements}] -> :Game



Graph DB Design

Gaming Twins Recommendation:

- **Domain-centric query:** Given a user, finds other users with highly similar game libraries, measured by the number of games they share.
- **Graph-centric query:** Given a user, find the user nodes that have games in common with the given user, and return the ones with the highest Jaccard similarity

```
MATCH (u1:User {id: $userId})

WITH u1, COUNT { (u1)-[:PLAYED]->(:Game) } AS u1Total

MATCH (u1)-[:PLAYED]->(g:Game)<-[:PLAYED]-(u2:User)

WITH u2, u1Total, count(g) AS sharedGames

WITH u2, u1Total, sharedGames,
    COUNT { (u2)-[:PLAYED]->(:Game) } AS u2Total

WITH u2, sharedGames,
        (u1Total + u2Total - sharedGames) AS
   unionGames

RETURN u2.id AS userId,
        u2.username AS username,
        u2.pfpURL AS pfpURL,
       toFloat(sharedGames) / unionGames AS
   jaccardSimilarity

ORDER BY jaccardSimilarity DESC

LIMIT $limit
```

Graph DB Design

Hidden Gems (Inverse Popularity)

- **Domain-centric query:** Given a user, finds niche titles that are popular within the user's social circle but have a low global player base.

- **Graph-centric query:** Given a user, this query traverses FRIENDS_WITH and PLAYED relationships to identify game nodes present in the user's friend nodes. It then filters out game nodes with a higher PLAYED relationship count than a given threshold.

```
MATCH (u:User {id: $userId})-[:FRIENDS_WITH]-(friend)
    -[:PLAYED]->(game:Game)

WHERE NOT (u)-[:PLAYED]->(game)

WITH game, count(DISTINCT friend) AS friendsPlaying

ORDER BY friendsPlaying DESC

LIMIT 50

MATCH (game)<-[:PLAYED]-(globalPlayer)

WITH game, friendsPlaying, count(globalPlayer) AS
    globalPopularity

WHERE globalPopularity < $nicheThreshold

RETURN game.id AS gameId, game.name AS name, game.
    image AS image,
        friendsPlaying AS friendsPlaying,
    globalPopularity AS globalPopularity

ORDER BY friendsPlaying DESC, globalPopularity ASC

LIMIT 10
```

Neo4j Indexes

USED: Unique property constraints on the **id** property for :User and :Game nodes

REASON: All graph queries require an "anchor" node. Without indexes, Neo4j performs a *NodeByLabelScan*, which scales poorly as the dataset grows.

OPTIMIZATION: Constraints enable *NodeIndexSeek*, providing near-instant entry points for complex traversals.

Index	State	Leading operator	DB hits	Time (ms)
:User(id)	without	NodeByLabelScan	15,002	40.6
	with	NodeIndexSeek	2	3.5
:Game(id)	without	NodeByLabelScan	122,612	151.2
	with	NodeIndexSeek	1	3.8

Handling Intra-DB Consistency

- **Application-Level Management**: Consistency between MongoDB and Neo4j is handled directly within the service layer.
- **Transactional Integrity**: Exploits Spring's @Transactional annotation to wrap operations spanning both databases into a single atomic execution block.
- **Unified Transactions**: Complex writes, such as user registration, catalog expansion, or establishing friendships, are treated as a single distributed unit.

Handling Intra-DB Consistency

Failure Handling & Atomic Rollback

- **Error Detection**: If a secondary operation fails (e.g., a Neo4j insertion error or a failed consistency check), a RuntimeException is triggered.
- **Automatic Rollback**: The system aborts the entire sequence, automatically rolling back any preceding MongoDB updates to prevent partial data states.
- **Structural Synchronization**: Guarantees that the document store and the graph database remains perfectly synchronized without requiring complex external retry mechanisms.

Swagger UI REST APIs documentation

Swagger UI: http://localhost:8080/swagger-ui/index.html

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

UNIVERSITÀ DI PISA

CROSSLAB Innovation for industry 4.0

Live Demo with Postman